Deep RL Quant Trading Agents for Partially Observable MDPs

Prashanth Jaganathan[1], **Purvaja Narayana**[1], **Shrijan S Shetty**[1]

[1]Northeastern University, Boston, MA, USA
jaganathan.p@northeastern.edu, narayana.p@northeastern.edu, shetty.shri@northeastern.edu

Abstract

Financial markets are noisy and partially observable, making them a natural testbed for deep reinforcement learning (DRL). Existing DRL trading agents often assume full observability and rely on naive return-based rewards that ignore risk. We present a comparative study of Rainbow DQN, Proximal Policy Optimization (PPO), and an LSTM-augmented Actor-Critic for single-asset equity trading, benchmarked against an industry recognized buy-and-hold baseline. We formulate the task as a POMDP and compare two belief-state approximations: a finite sliding window of OHLCV features and technical indicators, and a decoupled LSTM encoder pretrained via supervised next-observation prediction. We further introduce an event-based reward that pays out only on profitable round-trips and penalizes high-volatility entries, avoiding the sparsity and misaligned incentives of dense portfolio-value rewards. On AMZN (trending market), two agents beat buy-and-hold: LSTM-A2C, PPO. On CHWY (declining market), PPO achieves higher returns compared to buy-and-hold, while LSTM-A2C and Rainbow DQN fail to decouple from the baseline. These results demonstrate that reward engineering and temporal state modeling are critical design axes for DRL-based trading systems.

GitHub Repository

Dataset (yfinance)

Introduction

Algorithmic trading has long attracted interest from both finance practitioners and the machine learning community. The core challenge is deciding when to buy, sell, or hold a financial asset to maximize risk-adjusted returns, a sequential decision-making problem under uncertainty. Classical approaches such as mean-reversion strategies, momentum signals, and rule-based technical analysis rely on hand-crafted heuristics that generalize poorly across market regimes and fail to adapt when the statistical properties of the market shift. More sophisticated model-based methods, including stochastic control and mean-variance portfolio optimization, depend on assumptions of stationarity and Gaussianity that financial time series systematically violate: returns exhibit fat tails, volatility clusters temporally, and correlations spike during crises, rendering variance an inadequate measure of risk and invalidating the distributional models on which these methods are built.

Deep reinforcement learning offers a principled alternative: instead of specifying how to trade, an agent learns a policy by interacting with a market environment and receiving feedback in the form of rewards. This paradigm has produced strong results in domains with similarly complex, high-dimensional dynamics, including games, robotics, and resource management. However, applying DRL to financial markets introduces two challenges that existing work largely sidesteps.

Partial observability. Market prices are driven by latent factors, including institutional order flow, macroeconomic expectations, and sentiment, that are not directly observed by the agent. A fixed sliding window of OHLCV data provides only a partial view of the true market state. Standard DRL formulations model this as a fully observable MDP, which is inconsistent with the underlying data-generating process and can lead to policies that are brittle under distributional shift (Hausknecht and Stone 2015).

Reward misspecification. The most common reward signal in trading agents is the single-step change in portfolio value (Yang et al. 2020; Kabbani and Duman 2022). This is a dense but noisy signal that encourages excessive trading, ignores round-trip transaction costs, and provides no direct incentive for risk control. Agents trained on such rewards often learn to churn positions rather than identify profitable entry and exit points.

We address both challenges in a unified framework and present a systematic comparison of three DRL architectures, Rainbow DQN, and Proximal Policy Optimization (PPO), alongside an LSTM-augmented Actor Critic agent, evaluated on real equity data. Our contributions are as follows:

1. POMDP formulation with learned temporal context: We explicitly model single-asset trading as a POMDP, acknowledging that the agent's observation is a lossy projection of the true market state. For the LSTM-based agent, we pretrain a recurrent network via supervised learning to predict future market observations, freeze its weights, and use its hidden state as a compressed temporal context that augments the agent's observations during

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

PPO policy optimization. This two-phase approach decouples temporal modeling from policy learning, avoiding the instability of end-to-end recurrent policy gradients.

2. Event-based reward formulation: We propose a reward scheme that assigns profit-proportional rewards only when a position is closed (capturing the round-trip return after commissions) and applies a small volatility-scaled penalty at the moment of entry (discouraging positions in high-risk regimes), with zero reward for holding. This aligns agent incentives directly with profitable trading rather than noisy per-step portfolio fluctuations.
3. Comprehensive empirical comparison: We provide a controlled comparison of value-based (Rainbow DQN) and policy-gradient (PPO, LSTM-A2C) agents across multiple equities, reward schemes, and feature representations, benchmarked against a buy-and-hold baseline on chronologically held-out validation and test data.

Background

Formulation as POMDP

We model single stock trading as a POMDP, formally defined by the tuple $(\mathcal{S}, \mathcal{A}, T, R, \Omega, \mathcal{O}, \gamma)$. The true market state $s_t \in \mathcal{S}$, which includes order-flow microstructure, latent regime variables, and macroeconomic conditions, satisfies the Markov property but is not accessible to the agent. The agent instead receives observations $o_t \in \Omega$ generated by a deterministic projection $\mathcal{O} : \mathcal{S} \to \Omega$ that discards all information outside a finite price window and a small set of engineered statistics. The transition function $T(s_{t+1} \mid s_t, a_t)$ and reward function $R(s_t, a_t)$ are defined over the hidden state; the agent must learn a policy $\pi : \Omega \to \Delta(\mathcal{A})$ that approximates the optimal belief-state policy without explicit belief tracking.

Observation. The agent receives $o_t = [\,P_t,\ \ I_t,\ \ \phi_t\,]$, where:

- $P_t \in [0,1]^{W\times 5}$: min-max normalized OHLCV sliding window (Open, High, Low, Close, Volume) over the past W trading days, flattened row-wise.
- $I_t \in \mathbb{R}^6$: technical indicators computed at step t, consisting of RSI, MACD, MACD signal, Bollinger Band upper distance, Bollinger Band lower distance, and ATR, each normalized relative to the current closing price.
- $\phi_t \in \{0,\,1\}$: binary position flag ($\phi_t = 1$ if holding shares, 0 otherwise).

With $W = 20$, the observation is $o_t \in \mathbb{R}^{107}$. Following (Hausknecht and Stone 2015), the sliding window serves as an approximate sufficient statistic for the belief $b_t = \Pr(s_t \mid o_{1:t},\, a_{1:t-1})$, with I_t partially compensating for the window truncation by summarizing longer-horizon dynamics (e.g., MACD's 26-day EMA, ATR's 14-day average).

Action. $a_t \in \mathcal{A} = \{0,\,1,\,2\}$, corresponding to Hold, Buy (100% of available cash), and Sell (100% of held shares).

Transition. The transition decomposes into two independent components:

- *Market dynamics*: the exogenous evolution of prices and regime variables, unknown to the agent and approximated through historical replay during training.
- *Portfolio dynamics*: deterministic and fully known. Executed Buy/Sell actions update the cash balance b_t and share count n_t with a proportional commission rate $\delta = 0.1\%$ applied to each trade.

Reward. We evaluate two reward formulations, both computed from the observable portfolio value $V_t^{\text{port}} = b_t + \phi_t \cdot n_t \cdot p_t$ where p_t is the closing price at step t.

a) Event-based reward. Reward is emitted only at trade boundaries:

$$r_t = \begin{cases} \dfrac{p_t(1-\delta)\ -\ p_{\text{entry}}(1+\delta)}{p_{\text{entry}}(1+\delta)} & a_t = \text{Sell},\ \phi_t = 1 \\ -\lambda \cdot \sigma_W(t) & a_t = \text{Buy},\ \phi_t = 0 \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

where p_{entry} is the price at which the position was opened, $\sigma_W(t)$ is the standard deviation of returns over the preceding W steps, and λ is a small risk-aversion coefficient.

b) Portfolio-value reward. A dense signal applied uniformly across all actions:

$$R_t = \left(V_{t+1}^{\text{port}} - V_t^{\text{port}}\right)\ +\ \mathbf{1}[t = T] \cdot \left(V_T^{\text{port}} - V_0^{\text{port}}\right) \tag{2}$$

where the first term provides a per-step portfolio change signal and the indicator $\mathbf{1}[t = T]$ activates a terminal bonus equal to the total episode profit $V_T^{\text{port}} - V_0^{\text{port}}$ at episode termination, included to mitigate the long-horizon credit assignment problem inherent in sparse trading rewards.

Discount factor. $\gamma = 0.99$ (tuned for some experiments).

Value-Based Methods

a) DQN (Mnih et al. 2015) approximates $Q^*(s,a)$ with a neural network, stabilizing training via an experience replay buffer to break temporal correlations and a periodically-synced target network to reduce non-stationarity in the TD target.

b) Double DQN (Van Hasselt, Guez, and Silver 2016) decouples action selection from evaluation to reduce overestimation bias:

$$y_t = r_t + \gamma \cdot Q_{\theta^-}\Big(s_{t+1},\ \arg\max_a Q_\theta(s_{t+1}, a)\Big) \tag{3}$$

c) Rainbow DQN (Hessel et al. 2018) integrates six extensions into a single agent:

i. Prioritized replay (Schaul et al. 2016) samples transitions proportional to TD error magnitude $p_t \propto |\delta_t|^\omega$, with importance-sampling weights $w_i = (N \cdot P(i))^{-\beta}$ to correct the resulting bias.

ii. Dueling architecture (Wang et al. 2016) decomposes Q-values into state-value and advantage streams sharing a common encoder:

$$Q_\theta(s,a) = V_\eta\big(f_\xi(s)\big) + A_\psi\big(f_\xi(s), a\big) - \frac{1}{|\mathcal{A}|}\sum_{a'} A_\psi\big(f_\xi(s), a'\big) \tag{4}$$

enabling the network to learn state values independently of the action taken.

iii. Multi-step returns (Sutton and Barto 2018) replace single-step bootstrapping with n-step truncated returns propagating reward signal faster at the cost of higher variance.

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} \tag{5}$$

iv. Distributional RL (C51) (Bellemare, Dabney, and Munos 2017) models the full return distribution using N_{atoms} probability masses on a fixed support spanning $[V_{\min}, V_{\max}]$, trained by minimizing the KL divergence against a projected Bellman target:

$$\mathcal{L} = D_{\mathrm{KL}}\Big(\Phi\, \hat{d}_t^{(n)} \parallel d_t \Big) \tag{6}$$

v. Noisy networks (Fortunato et al. 2018) replace ϵ-greedy exploration with learned parametric noise on network weights, $y = (\mu^w + \sigma^w \odot \varepsilon^w)x + (\mu^b + \sigma^b \odot \varepsilon^b)$, providing state-dependent exploration that self-anneals as training progresses.

Proximal Policy Optimization

PPO (Schulman et al. 2017) is an on-policy actor-critic algorithm that optimizes a clipped surrogate objective:

$$L^{\text{CLIP}} = \mathbb{E}\left[\min\Big(r_t \hat{A}_t,\ \text{clip}(r_t,\ 1-\epsilon,\ 1+\epsilon)\, \hat{A}_t\Big)\right] \tag{7}$$

where r_t is the probability ratio between the new and old policy. This prevents destructively large policy updates without requiring a hard trust region constraint. Advantages are estimated using Generalized Advantage Estimation (GAE) with $\gamma = 0.99$ and $\lambda = 0.95$.

Related Work

Policy Gradient and Actor-Critic Methods.

Beyond PPO, methods such as A3C, SAC, and TD3 have shown strong performance on continuous control tasks. SAC and TD3 are designed for continuous action spaces, making them less directly applicable to our discrete Buy/Hold/Sell formulation without modification. A3C uses asynchronous parallel workers to improve sample efficiency but introduces implementation complexity without a clear advantage on single-asset episodic trading. PPO offers a strong balance of stability, sample efficiency, and simplicity, making it the natural policy gradient choice.

Recurrent and Attention-Based Policies.

End-to-end recurrent architectures such as LSTM-PPO train the LSTM jointly with the policy, but this is known to be unstable due to exploding gradients and sensitivity to initialization. Transformer-based approaches apply self-attention over price sequences and have shown promise but require significantly more data and computation. Our POMDP agent avoids end-to-end recurrent training by pretraining the LSTM as a supervised dynamics model and freezing it, providing stable temporal context without joint optimization instability.

Description

Trading Environment

We implement a custom single-asset trading environment following the Gymnasium interface. The environment simulates realistic discrete-time trading over historical equity price data with transaction costs, position constraints, and pluggable reward schemes.

Data and Splits Historical daily OHLCV data is downloaded via `yfinance` for four equities: MSFT, AAPL, AMZN, and CHWY. Technical indicators are precomputed over the full price series prior to any splitting. The data is partitioned chronologically into train, validation, and test sets to prevent lookahead bias, with each split instantiated as an independent environment.

Observation Space The observation at timestep t is a fixed-length vector $s_t \in \mathbb{R}^{107}$ constructed by the `OHLCVWithIndicators` feature builder:

$$s_t = \left[\tilde{P}_t^{\text{flat}} \parallel I_t \parallel \phi_t\right] \tag{8}$$

where $\tilde{P}_t^{\text{flat}} \in [0,1]^{100}$ is the min–max normalized OHLCV sliding window of $W = 20$ timesteps flattened row-wise, $I_t \in \mathbb{R}^6$ is a vector of normalized technical indicators (RSI, MACD, MACD signal, Bollinger Band distances, ATR), and $\phi_t \in \{0,1\}$ is a binary position flag indicating whether the agent currently holds shares.

Feature Construction The OHLCV window P_t is normalized per column within the sliding window using min–max scaling:

$$\tilde{P}_t^{(c)} = \frac{P_t^{(c)} - \min_w\big(P^{(c)}\big)}{\max_w\big(P^{(c)}\big) - \min_w\big(P^{(c)}\big) + \epsilon} \tag{9}$$

The technical indicator vector $I_t \in \mathbb{R}^6$ is precomputed over the full price series and normalized relative to the current close price p_t.

Action Space and Masking The action space is discrete: $\mathcal{A} = \{0,1,2\}$ corresponding to HOLD, BUY, and SELL respectively. To enforce physically valid transitions, invalid actions are hard-constrained by setting their logits to $-\infty$ before the action distribution is constructed. A boolean action mask is computed at each step from the agent's current position:

$$\text{mask}(a) = \begin{cases} True & a = \text{HOLD} \\ True & a = \text{BUY},\ \phi_t = 0 \\ True & a = \text{SELL},\ \phi_t = 1 \\ False & \text{otherwise} \end{cases} \tag{10}$$

This prevents the agent from buying while already holding a position or selling when flat.

Trade Execution The agent begins each episode with an initial cash balance of $B_0 = \$10{,}000$ and zero shares. A commission rate of $\delta = 0.001$ (0.1%) is applied to both legs of a trade.

Buy. The agent purchases the maximum affordable integer number of shares:

$$n_t = \left\lfloor \frac{B_t}{p_t(1+\delta)} \right\rfloor, \qquad B_{t+1} = B_t - n_t \cdot p_t(1+\delta) \quad (11)$$

Sell. All held shares are liquidated at the current price net of commission:

$$B_{t+1} = B_t + n_t \cdot p_t(1-\delta), \qquad n_{t+1} = 0 \quad (12)$$

Portfolio value at any timestep is defined as:

$$V_t^{\text{port}} = B_t + n_t \cdot p_t \quad (13)$$

Reward Functions Two reward formulations are evaluated:

a) Event-based reward. Reward is emitted only at trade boundaries, as defined in Equation 1. Sell actions receive the commission-adjusted round-trip return, Buy actions incur a small penalty proportional to recent volatility σ_W, and Hold actions receive zero reward. This formulation directly incentivizes profitable round-trips while penalizing entries during high-volatility periods.

b) Portfolio-value reward. Each step receives the change in portfolio value $V_{t+1}^{\text{port}} - V_t^{\text{port}}$, with a terminal bonus $V_T^{\text{port}} - V_0^{\text{port}}$ added at episode termination to aid long-horizon credit assignment, as defined in Equation 2.

Episode Initialization and Termination To prevent the agent from overfitting to a fixed market segment, each episode begins at a uniformly sampled starting index:

$$t_0 \sim \mathcal{U}\big[W,\ \max(W,\ |D| - T_{\max})\big] \quad (14)$$

where $|D|$ is the length of the price series and $T_{\max} = 252$ is the maximum episode length (approximately one trading year). An episode terminates when either the end of the dataset is reached or $T_{\max}$ steps have elapsed:

$$d_t = \mathbf{1}[t \geq |D| - 1] \ \vee \ \mathbf{1}[t - t_0 \geq T_{\max}] \quad (15)$$

Evaluation Metrics All agents are evaluated on held-out test data using greedy action selection ($\arg\max$ over masked logits) and benchmarked against a buy-and-hold baseline that purchases on the first step and holds until termination.

Algorithms

1. Rainbow DQN

Rainbow DQN (Hessel et al. 2018) integrates six improvements over vanilla DQN into a unified agent. We describe our implementation of each component and their interaction in the trading domain.

Network Architecture: Dueling Distributional Noisy Net The Rainbow agent maintains two identically-structured networks: an online network Q_θ and a target network Q_{θ^-}. Rather than outputting scalar Q-values, each network outputs a probability distribution over returns for every action, parameterized on a fixed discrete support.

The support consists of $N_{\text{atoms}} = 51$ evenly-spaced atoms spanning $[V_{\min},\ V_{\max}]$:

$$\begin{aligned} z_i &= V_{\min} + (i-1) \cdot \Delta z, \\ \Delta z &= \frac{V_{\max} - V_{\min}}{N_{\text{atoms}} - 1}, \\ i &= 1, \ldots, N_{\text{atoms}} \end{aligned} \quad (16)$$

The network first maps the observation through a shared feature encoder:

$$f = \text{ReLU}(W_f\, o_t + b_f), \quad W_f \in \mathbb{R}^{128 \times d} \quad (17)$$

where d is the observation dimension. The output then splits into two streams following the dueling architecture (Wang et al. 2016):

- **Value stream**: NoisyLinear$(128, 128)$ $\rightarrow$ ReLU $\rightarrow$ NoisyLinear$(128, N_{\text{atoms}})$, producing per-atom state values $V_\eta^i(f) \in \mathbb{R}^{N_{\text{atoms}}}$.
- **Advantage stream**: NoisyLinear$(128, 128)$ $\rightarrow$ ReLU $\rightarrow$ NoisyLinear$(128, |\mathcal{A}| \times N_{\text{atoms}})$, producing per-atom advantages $A_\psi^i(f, a) \in \mathbb{R}^{|\mathcal{A}| \times N_{\text{atoms}}}$.

The streams are aggregated per atom and normalized via softmax to yield a valid probability mass function over atoms for each action:

$$p_\theta^i(s, a) = \frac{\exp\big(V_\eta^i(f) + A_\psi^i(f, a) - \bar{A}_\psi^i(f)\big)}{\sum_j \exp\big(V_\eta^j(f) + A_\psi^j(f, a) - \bar{A}_\psi^j(f)\big)} \quad (18)$$

where $\bar{A}_\psi^i(f) = \frac{1}{|\mathcal{A}|} \sum_{a'} A_\psi^i(f, a')$. The expected Q-value used for action selection is recovered as:

$$Q_\theta(s, a) = \sum_{i=1}^{N_{\text{atoms}}} z_i \cdot p_\theta^i(s, a) \quad (19)$$

Noisy Linear Layers All four hidden-to-output linear layers in the value and advantage streams are replaced with factorized Gaussian noisy layers (Fortunato et al. 2018), eliminating the need for ϵ-greedy exploration ($\epsilon = 0$ throughout). Each NoisyLinear layer computes:

$$y = (\mu^w + \sigma^w \odot \varepsilon^w)\, x + (\mu^b + \sigma^b \odot \varepsilon^b) \quad (20)$$

where μ^w, σ^w are learned weight parameters, and $\varepsilon^w = \varepsilon_{\text{out}}\, \varepsilon_{\text{in}}^\top$ is factorized noise constructed from per-input and per-output noise vectors via the signed-square-root transform $f(x) = \text{sgn}(x)\sqrt{|x|}$. Parameters are initialized as $\mu \sim \mathcal{U}(-1/\sqrt{d_{\text{in}}},\ 1/\sqrt{d_{\text{in}}})$ and $\sigma_0 = 0.5/\sqrt{d_{\text{in}}}$. During evaluation, noise is disabled and only μ is used. Noise is resampled once per forward pass during training, providing state-dependent exploration that self-anneals as σ shrinks.

Multi-Step Returns Instead of storing single-step transitions, incoming transitions are passed through an n-step buffer with $n = 3$. The buffer maintains a rolling window of the most recent n transitions and, once full, emits a collapsed n-step transition:

$$\big(s_t,\ a_t,\ R_t^{(n)},\ s_{t+n},\ d_{t+n}\big), \quad R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k\, r_{t+k+1} \quad (21)$$

If a terminal state is encountered before n steps, the return is truncated at the terminal transition and all remaining partial windows are flushed. The n-step buffer is cleared at episode boundaries to prevent reward leakage across episodes.

Prioritized Experience Replay The emitted n-step transitions are stored in a prioritized replay buffer (Schaul et al. 2016) of capacity 50,000, implemented via a binary sum-tree for $O(\log N)$ sampling. Each transition i is sampled with probability:

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}, \quad p_i = |\mathcal{L}_i| + \varepsilon \tag{22}$$

where $\mathcal{L}_i$ is the per-sample KL loss (rather than the scalar TD error used in standard PER), $\alpha = 0.5$ controls the degree of prioritization, and $\varepsilon = 10^{-6}$ ensures non-zero sampling probability. New transitions enter with the current maximum priority to guarantee they are sampled at least once.

To correct the bias introduced by non-uniform sampling, each transition's loss is weighted by its importance-sampling weight:

$$w_i = \frac{(N \cdot P(i))^{-\beta}}{\max_j (N \cdot P(j))^{-\beta}} \tag{23}$$

where β is linearly annealed from $\beta_0 = 0.4$ to 1.0 over the course of training to ensure full bias correction by convergence.

Distributional Target Computation Each gradient step proceeds as follows. A minibatch $\mathcal{B}$ of size 64 is drawn from the prioritized buffer. The online network computes the current return distribution $d_t = (z,\, p_\theta(s_t, a_t))$. The target distribution is constructed using double Q-learning action selection combined with distributional projection.

First, the online network selects the greedy action in the next state via its expected Q-values:

$$a^* = \arg\max_{a'} Q_\theta(s_{t+n},\, a') \tag{24}$$

The target network then provides the return distribution for this action: $p_{\theta^-}(s_{t+n}, a^*)$. The n-step distributional Bellman target shifts and contracts each atom of this distribution:

$$\hat{\mathcal{T}} z_i = \text{clip}\Big(R_t^{(n)} + \gamma^n z_i \cdot (1 - d_{t+n}),\; V_{\min},\; V_{\max}\Big) \tag{25}$$

Since the projected atoms $\hat{\mathcal{T}} z_i$ generally do not align with the fixed support, each is distributed to its two nearest neighbors via ℓ_2-projection:

$$\begin{aligned} b_i &= \frac{\hat{\mathcal{T}} z_i - V_{\min}}{\Delta z}, \\ m_{\lfloor b_i \rfloor} &\mathrel{+}= p_{\theta^-}^i \cdot (\lceil b_i \rceil - b_i), \\ m_{\lceil b_i \rceil} &\mathrel{+}= p_{\theta^-}^i \cdot (b_i - \lfloor b_i \rfloor) \end{aligned} \tag{26}$$

where $m \in \mathbb{R}^{N_{\text{atoms}}}$ is the projected target distribution.

Loss and Optimization The loss is the importance-weighted cross-entropy between the projected target m and the current distribution:

$$\mathcal{L}_i = -\sum_{j=1}^{N_{\text{atoms}}} m_j \log p_\theta^j(s_t, a_t), \qquad \mathcal{L}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} w_i \cdot \mathcal{L}_i \tag{27}$$

This is equivalent to minimizing $D_{\text{KL}}(\Phi\, \hat{d}_t^{(n)} \| d_t)$ up to a constant. After each gradient step, the per-sample losses $\mathcal{L}_i$ are used to update the priorities in the sum-tree. Gradients are clipped to a maximum norm of 10.0, and the network is optimized with Adam at learning rate 10^{-4} with $\varepsilon_{\text{Adam}} = 1.5 \times 10^{-4}$.

Target Network Synchronization The target network Q_{θ^-} is synchronized via hard copy from the online network every $C = 500$ gradient steps:

$$\theta^- \leftarrow \theta \qquad \text{every } C \text{ gradient steps} \tag{28}$$

Reward Structure The Rainbow agent uses the same portfolio-value reward as the DQN agent, defined in Equation 2.

Algorithm 1: Rainbow DQN for Trading

Require: Q_θ, Q_{θ^-} randomly initialized
  $\theta^- \leftarrow \theta$
  $\mathcal{D}_{\text{PER}} \leftarrow \emptyset$, $\mathcal{N}_{\text{step}} \leftarrow \emptyset$
  step $\leftarrow 0$, $\beta \leftarrow 0.4$
  for each episode **do**
    $o_0 \leftarrow$ `env.reset()`
    for $t = 0$ to $T - 1$ **do**
      Q_θ`.reset_noise()`
      $a_t \leftarrow \arg\max_a Q_\theta(o_t, a)$
      $o_{t+1}, r_t, d_t \leftarrow$ `env.step`(a_t)
      Push $(o_t, a_t, r_t, o_{t+1}, d_t)$ into $\mathcal{N}_{\text{step}}$
      for each emitted n-step transition $(s, a, R^{(n)}, s_n, d_n)$ **do**
        $\mathcal{D}_{\text{PER}}$`.push`$(s, a, R^{(n)}, s_n, d_n)$
      end for
      step $\leftarrow$ step $+ 1$
      if $|\mathcal{D}_{\text{PER}}| \geq 1000$ **and** step $\bmod 4 = 0$ **then**
        Sample $\mathcal{B} \sim \mathcal{D}_{\text{PER}}$
        Compute IS-weights w_i
        Q_θ`.reset_noise()`, Q_{θ^-}`.reset_noise()`
        Compute projected target m (Eq. 14–16)
        $\mathcal{L}(\theta) \leftarrow \frac{1}{|\mathcal{B}|} \sum_i w_i \mathcal{L}_i$
        Update θ (Adam), clip $\|\nabla\| \leq 10$
        $p_i \leftarrow (|\mathcal{L}_i| + \varepsilon)^\alpha$
        $\beta \leftarrow \min\Big(1,\, \beta_0 + \frac{\text{step}(\beta_1 - \beta_0)}{\text{anneal_steps}}\Big)$
        if step $\bmod 500 = 0$ **then**
          $\theta^- \leftarrow \theta$
        end if
      end if
    end for
    Flush $\mathcal{N}_{\text{step}}$
  end for

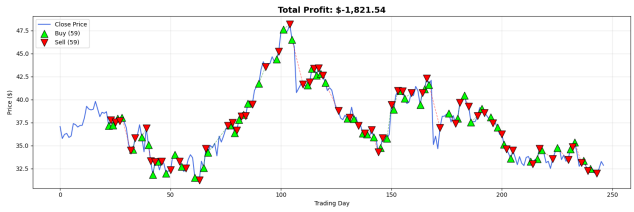

Figure 1: Rainbow DQN Agent trading behavior on Test dataset for CHWY (-18% return)

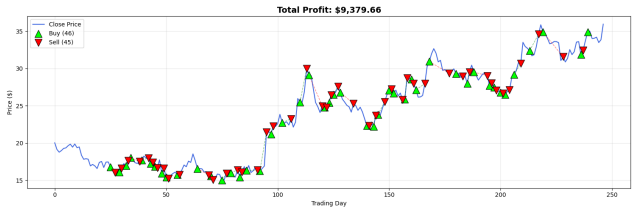

Figure 2: Rainbow DQN Agent trading behavior on Validation dataset for CHWY (+93% return)

2. PPO

Network Architecture The PPO agent maintains two independent feedforward networks. The policy network π_θ maps observations to action logits:

$$\pi_\theta : \mathbb{R}^{107} \xrightarrow{\text{Linear}(128)} \text{ReLU} \xrightarrow{\text{Linear}(128)} \text{ReLU} \xrightarrow{\text{Linear}(3)} \boldsymbol{\ell}_t \tag{29}$$

The value network V_ϕ shares an identical architecture with output dimension 1, producing a scalar state-value estimate. Both networks are optimized jointly by a single Adam optimizer with learning rate 3×10^{-4}.

Rollout Collection At each training iteration the agent collects a fixed-length rollout of $N = 64$ timesteps by interacting with the environment. Each transition $\big(s_t,\, a_t,\, \log \pi_\theta(a_t \mid s_t),\, r_t,\, V_\phi(s_t),\, d_t\big)$ is stored in a rollout buffer. Actions are sampled as $a_t \sim \text{Categorical}(\tilde{\boldsymbol{\ell}}_t)$, where $\tilde{\boldsymbol{\ell}}_t$ are the masked logits from Equation (7).

Generalized Advantage Estimation

After each rollout, advantages are computed via GAE. The TD residual at each timestep is:

$$\delta_t = r_t + \gamma\, V_\phi(s_{t+1})(1 - d_t) - V_\phi(s_t) \tag{30}$$

The advantage estimate accumulates discounted residuals backward through time:

$$\hat{A}_t = \sum_{l=0}^{N-t-1} (\gamma\lambda)^l\, \delta_{t+l} \tag{31}$$

with $\gamma = 0.99$, $\lambda = 0.95$, and terminal bootstrap $V_\phi(s_T) = 0$. Value targets are $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$. Advantages are normalized across the rollout before the update:

$$\hat{A}_t \leftarrow \frac{\hat{A}_t - \mu_{\hat{A}}}{\sigma_{\hat{A}} + \epsilon} \tag{32}$$

Policy Update

For $K = 10$ epochs, minibatches of size 64 are sampled uniformly from the rollout buffer. The importance sampling ratio between the updated and old policy is:

$$r_t(\theta) = \exp\big(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)\big) \tag{33}$$

The clipped surrogate objective prevents destructively large policy updates:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta),\, 1{-}\epsilon,\, 1{+}\epsilon)\,\hat{A}_t\big)\Big], \tag{34}$$

The value and entropy losses are:

$$L^{\text{VF}}(\phi) = \mathbb{E}_t\Big[\big(V_\phi(s_t) - \hat{R}_t\big)^2\Big] \tag{35}$$

$$\mathcal{H}(\pi_\theta) = -\mathbb{E}_t\left[\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t)\right] \tag{36}$$

The combined loss minimized at each gradient step is:

$$\mathcal{L}(\theta, \phi) = -L^{\text{CLIP}}(\theta) + 0.5 \cdot L^{\text{VF}}(\phi) - 0.01 \cdot \mathcal{H}(\pi_\theta) \tag{37}$$

Gradients are clipped to a maximum ℓ_2 norm of 0.5 before each parameter update.

Algorithm 2: PPO for Trading

Require: TradingEnv, $M = 500$ episodes, $N = 64$ rollout steps
Initialize: π_θ, V_ϕ randomly; Adam optimizer, lr $= 3 \times 10^{-4}$
for $m = 1$ **to** M **do**
  $s_0 \leftarrow$ `env.reset()` ▷ Random start t_0
  for $t = 0$ **to** $N - 1$ **do**
    $\text{mask}_t \leftarrow$ `env.get_action_mask()`
    $a_t \sim \text{Categorical}\big(\pi_\theta(\cdot \mid s_t),\ \text{mask}_t\big)$
    $s_{t+1},\, r_t,\, d_t \leftarrow$ `env.step`(a_t)
    `buffer.store`$\big(s_t,\, a_t,\, \log \pi_\theta(a_t \mid s_t),\, r_t,\, V_\phi(s_t),\, d_t\big)$
  end for
  Compute δ_t, $\hat{A}_t$ via GAE ($\gamma = 0.99,\ \lambda = 0.95$)
  Normalize $\hat{A}_t$; compute targets $\hat{R}_t$
  for $k = 1$ **to** $K = 10$ **do**
    for each minibatch $\mathcal{B}$ ($|\mathcal{B}| = 64$) **in** shuffled buffer **do**
      Compute $r_t(\theta)$, L^{CLIP}, L^{VF}, $\mathcal{H}$
      Update θ, ϕ via Adam on $\mathcal{L}(\theta, \phi)$ with grad clip 0.5
    end for
  end for
  Clear buffer
end for

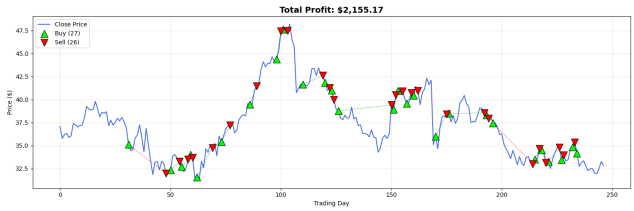

Figure 3: PPO agent trading behavior on Test dataset for CHWY

Figure 4: PPO agent trading behavior on Validation dataset for CHWY

Figure illustrates the PPO agent's trading behavior across training and validation on CHWY. At Episode 50, the agent executes 58 round-trips concentrated in a sharply declining segment (days 650–900), incurring a loss of $4,795. By Episode 150, trade frequency drops to 41 round-trips and activity shifts toward the peak region (days 400–600), reducing losses to $723. This reflects the policy gradient updates progressively discouraging entries during sustained downtrends. By Episode 250, the agent achieves its first profitable episode ($2,516), concentrating trades around local price troughs near days 700–800. The progression demonstrates PPO converging from erratic overtrading toward more selective, risk-aware entry and exit decisions.

3. LSTM based POMDP Agent

Problem Formulation and Observability

The POMDP Reality. In real-world financial markets, an algorithmic trading agent cannot observe the true global state s_t. The true state incorporates latent variables such as institutional order flow, private investor sentiment, and global macroeconomic shifts. Instead, the agent receives only a limited local observation $o_t \in \Omega$ drawn deterministically from an observation function $\mathcal{O}(o_t \mid s_t)$. Because o_t omits these broader market factors, it violates the Markov property, the immediate observation alone is insufficient to optimally predict future price transitions. Consequently, we formulate the trading environment as a POMDP defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma)$.

Baseline Mitigation: Windowed Observations. A standard approach to navigating a POMDP in Deep Reinforcement Learning is to expand the immediate observation into a short historical sequence. In our baseline algorithms, we mitigate partial observability by defining o_t as a sliding window of recent market states (e.g., an $o_t \in \mathbb{R}^{107}$ vector representing the past W days of OHLCV prices and technical indicators). By concatenating recent frames, the baseline agents receive a synthetic, short-term temporal context. However, this approach creates a rigid computational cutoff. While it captures localized momentum, it fundamentally acts as a naive approximation of the true POMDP belief state and fails to encode regimes that extend beyond the fixed window size.

Our Advanced Implementation: Recurrent Belief State Approximation. To rigorously address the POMDP formulation beyond a fixed window, an agent must infer a belief state over all possible true states s_t given its entire history of observations $\{o_1, \ldots, o_t\}$. In our Partial MDP (PMDP) architecture, we approximate this temporal sufficient statistic using a decoupled Recurrent Neural Network. Specifically, we employ a Long Short-Term Memory (LSTM) encoder (Hochreiter and Schmidhuber 1997). Even though the LSTM is still fed the localized window o_t at each timestep, it recursively updates an internal hidden state vector h_t:

$$h_t = \text{LSTM}(o_t, h_{t-1}) \tag{38}$$

By continuously compressing the trajectory of past observations into h_t, the LSTM acts as a robust, empirical belief state. This temporal context is passed continuously to the Proximal Policy Optimization (PPO) Actor-Critic network. Thus, by conditioning the policy $\pi(a_t \mid o_t, h_t)$ on both the immediate windowed observation and the inferred continuous timeline, the agent is freed from the limitations of the fixed window and can navigate the POMDP based on long-term market regimes.

Architecture The POMDP agent consists of two components: a frozen LSTM predictor that encodes temporal market context, and a PPO actor-critic that conditions on the concatenation of the current observation and the LSTM hidden state.

LSTM Predictor. The predictor f_ψ is a single-layer LSTM that maps market observations (excluding the position flag) to a hidden state:

$$h_t, c_t = \text{LSTM}_\psi\big(o_t^{\text{mkt}},\ h_{t-1},\ c_{t-1}\big) \tag{39}$$

where $o_t^{\text{mkt}} \in \mathbb{R}^{106}$ excludes the position scalar ϕ_t. The LSTM has hidden dimension 128. A linear output head projects h_t back to the market observation space for the pretraining objective:

$$\hat{o}_{t+1}^{\text{mkt}} = W_{\text{out}}\, h_t + b, \quad W_{\text{out}} \in \mathbb{R}^{106\times 128} \tag{40}$$

Actor-Critic Network. The augmented state fed to the actor-critic is the concatenation of the current observation and the LSTM hidden state:

$$\tilde{s}_t = [o_t \parallel h_t] \in \mathbb{R}^{107+128} = \mathbb{R}^{235} \tag{41}$$

The actor and critic are independent two-layer MLPs operating on $\tilde{s}_t$:

$$\pi_\theta(\tilde{s}_t) = W_3\, \text{ReLU}\big(W_2\, \text{ReLU}(W_1 \tilde{s}_t)\big) \ \rightarrow\ \text{logits} \in \mathbb{R}^3 \tag{42}$$

$$V_\phi(\tilde{s}_t) = W_3'\, \text{ReLU}\big(W_2'\, \text{ReLU}(W_1' \tilde{s}_t)\big) \ \rightarrow\ \mathbb{R} \tag{43}$$

where $W_1, W_1' \in \mathbb{R}^{128\times 235}$ and $W_2, W_2' \in \mathbb{R}^{128\times 128}$.

Phase 1: Supervised LSTM Pretraining The LSTM is pretrained to predict the next market observation given the current hidden state. For $E_{\text{pre}} = 50$ episodes, trajectories are collected under a uniformly random policy (sampling only from valid actions per the action mask). Given an episode trajectory $\{o_0, o_1, \ldots, o_T\}$, the LSTM is trained via BPTT to minimize the one-step-ahead prediction loss:

$$\mathcal{L}_{\text{pre}}(\psi) = \frac{1}{T} \sum_{t=0}^{T-1} \left\| \hat{o}_{t+1}^{\text{mkt}} - o_{t+1}^{\text{mkt}} \right\|_2^2 \tag{44}$$

Each episode trajectory is treated as a single sequence with batch size 1. The LSTM optimizer uses Adam with learning rate 10^{-3} and gradient clipping at norm 0.5. After pretraining completes, all LSTM weights ψ are frozen via `requires_grad=False`.

Phase 2: PPO with Frozen LSTM With the LSTM frozen, the actor-critic is trained using PPO over $M = 500$ episodes. The hidden state h_t is propagated explicitly across timesteps and reset to $\mathbf{0}$ at each episode boundary. At every step, the LSTM is stepped forward under `torch.no_grad()` to update h_t without accumulating gradients:

$$h_t = f_\psi^h\big(o_{t-1},\ h_{t-1}\big) \qquad \text{(no gradient)} \tag{45}$$

The stored hidden state h_t is used in the minibatch update to re-evaluate the actor-critic on transitions, avoiding the need to re-run the LSTM during the policy update. The rollout buffer therefore stores the tuple $(o_t,\ h_t,\ a_t,\ \log \pi_\theta,\ r_t,\ V_\phi,\ d_t)$.

GAE, advantage normalization, clipped surrogate objective, value loss, and entropy bonus are identical to PPO. The combined loss is:

$$\mathcal{L}(\theta, \phi) = -L^{\text{CLIP}}(\theta) + 0.5 \cdot L^{\text{VF}}(\phi) - 0.01 \cdot \mathcal{H}(\pi_\theta) \tag{46}$$

Algorithm 3: POMDP Agent Training

— Phase 1: LSTM Pretraining —
Initialize: LSTM_ψ; Adam, lr $= 10^{-3}$
for $e = 1$ **to** $E_{\text{pre}} = 50$ **do**
  $o_0 \leftarrow$ `env.reset()`; $h_0 = \mathbf{0}$
  Collect episode $\{o_0, \ldots, o_T\}$ under random valid policy
  Minimize $\mathcal{L}_{\text{pre}}(\psi)$ via BPTT, grad clip ≤ 0.5
end for
Freeze ψ

— Phase 2: PPO Actor-Critic —
Initialize: π_θ, V_ϕ; Adam, lr $= 3 \times 10^{-4}$
for $m = 1$ **to** $M = 500$ **do**
  $o_0 \leftarrow$ `env.reset()`; $h_0 = \mathbf{0}$
  for $t = 0$ **to** $N{-}1 = 63$ **do**
    $\tilde{s}_t \leftarrow [o_t \parallel h_t]$
    $a_t \sim \text{Categorical}\big(\pi_\theta(\tilde{s}_t),\ \text{mask}_t\big)$
    $o_{t+1},\ r_t,\ d_t \leftarrow$ `env.step`(a_t)
    `buffer.store`$\big(o_t,\ h_t,\ a_t,\ \log \pi_\theta,\ r_t,\ V_\phi(\tilde{s}_t),\ d_t\big)$
    $h_{t+1} \leftarrow f_\psi^h(o_t,\ h_t)$ `[no_grad]`; reset if d_t
  end for
  Compute $\hat{A}_t$, $\hat{R}_t$ via GAE (γ=0.99, λ=0.95)
  for $k = 1$ **to** 10 **do**
    for each minibatch $\mathcal{B}$ ($|\mathcal{B}|$=64) **do**
      Re-evaluate $\pi_\theta(\tilde{s})$, $V_\phi(\tilde{s})$ using stored h_t
      Update θ, ϕ on $\mathcal{L}(\theta, \phi)$, clip grad ≤ 0.5
    end for
  end for
  Clear buffer
end for

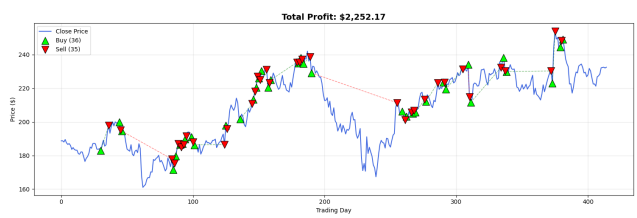

Figure 5: POMDP agent trading behavior on Test dataset for CHWY

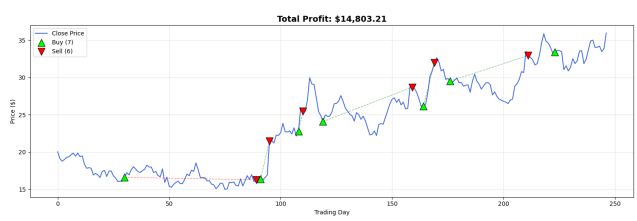

Figure 6: POMDP agent trading behavior on Validation dataset for CHWY

Figure 3 shows the agent's trading behavior on the val-

idation and test sets. On the test set (top), the agent executes only 4 buys and 3 sells across 250 trading days, reflecting highly selective decision-making. However, several entries occur ahead of local price declines, resulting in a net loss of $631. On the validation set (bottom), the agent identifies a prolonged uptrend beginning near day 90, entering at a price trough of approximately $16 and executing 7 well-timed round-trips across the subsequent rally, yielding a profit of $14,803. The contrast highlights strong validation performance but limited generalization to unseen test market conditions.

Experiments

We evaluate all three agents on two equities — AMZN and CHWY across validation and test splits, benchmarked against a buy-and-hold baseline.

Datasets Daily OHLCV data for AMZN, AAPL, MSFT, and CHWY was obtained via `yfinance` and split chronologically into train, validation, and test sets to prevent lookahead bias. These tickers span diverse volatility profiles—AMZN and AAPL represent large-cap growth equities, while CHWY provides a smaller, higher-volatility counterpart. Technical indicators are precomputed over the full price series prior to splitting, as described in Section **??**.

Evaluation Protocol All agents are evaluated on held-out splits using greedy action selection $\epsilon = 0$ for DQN and $\arg\max$ over masked logits for PPO and LSTM-PPO over a single full-length episode with no step limit. Performance is reported as cumulative return $(V_T^{\text{port}} - V_0^{\text{port}})/V_0^{\text{port}}$ and benchmarked against a buy-and-hold baseline that purchases on the first trading day and holds until termination.

Implementation Details All agents are implemented in PyTorch with Gymnasium environments, with an initial portfolio balance of $10,000 and commission rate $\delta = 0.001$. PPO and LSTM-PPO use the event-based reward scheme; DQN uses the portfolio-value reward during training. LSTM-PPO undergoes 50 supervised pretraining episodes before RL training begins.

Results

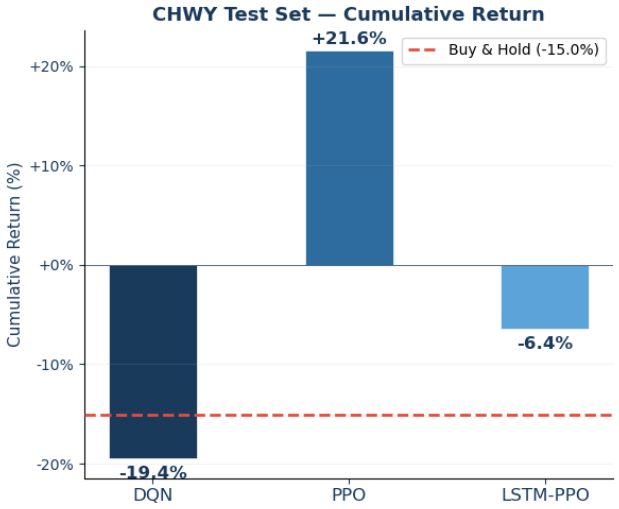

Figure 7: Cumulative Rewards on CHEWY on Test dataset

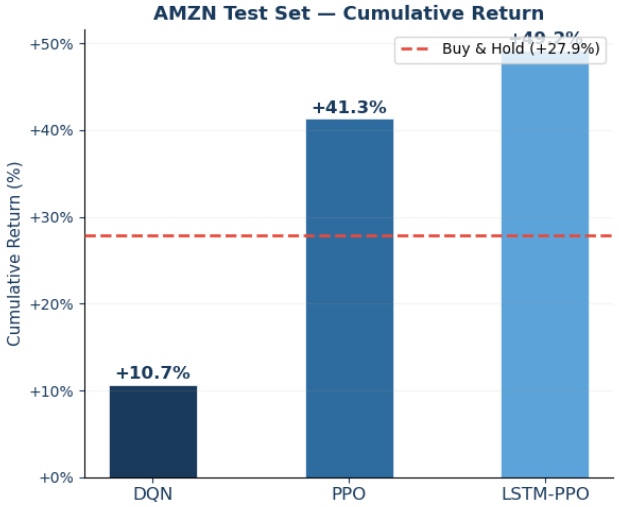

Figure 8: Cumulative Rewards on AMAZON on Test dataset

Test Set Performance: On the AMZN test set, all three agents outperform buy-and-hold (+27.9%). LSTM-PPO achieves the highest cumulative return at +49.3%, followed by PPO at +41.3% and DQN at +10.7%. This demonstrates that temporal belief approximation via LSTM provides a meaningful advantage on a trending equity, with LSTM-PPO outperforming vanilla PPO by 8 percentage points.

On the CHWY test set, the market declines over the evaluation period (buy-and-hold: -15.0%), making capital preservation the key challenge. PPO achieves +21.6%, substantially outperforming buy-and-hold by 36.6 percentage points. LSTM-PPO returns -6.4%, beating buy-and-hold by 8.6 points but lagging behind PPO. DQN performs worst at -19.4%, failing to preserve capital.

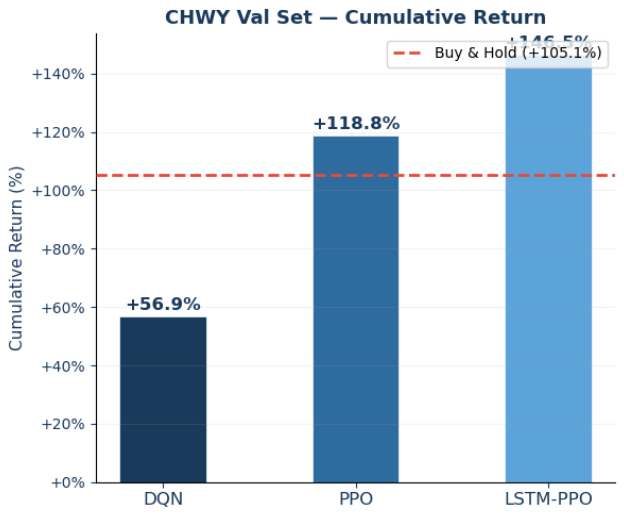

Figure 9: Cumulative Rewards on CHEWY on Validation dataset

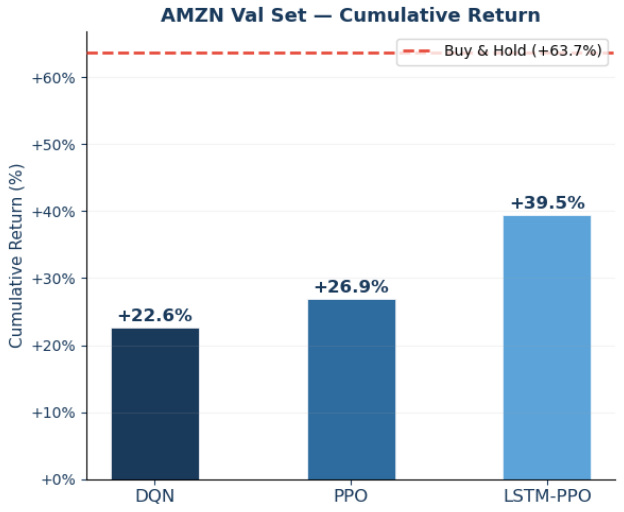

Figure 10: Cumulative Rewards on AMAZON on Validation dataset

On Validation Dataset: On AMZN validation, LSTM-PPO leads at +39.5%, followed by PPO at +26.9% and DQN at +22.6%, though all three agents underperform the buy-and-hold baseline of +63.7%, reflecting a strongly trending bull market that passive holding captures most effectively.

On CHWY validation, all agents achieve strong absolute returns. LSTM-PPO reaches +146.5% and PPO +118.8%, both exceeding the buy-and-hold baseline of +105.1%. DQN returns +56.9%, below the baseline.

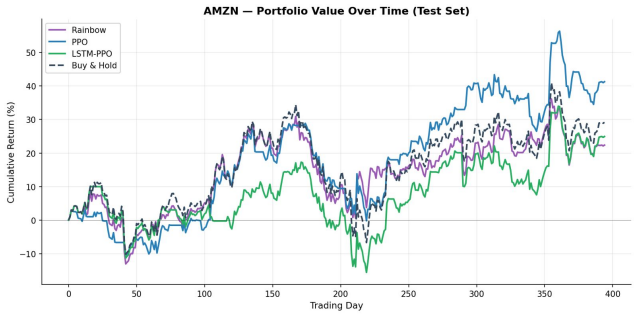

Figure 11: Portfolio Over Time for AMAZON

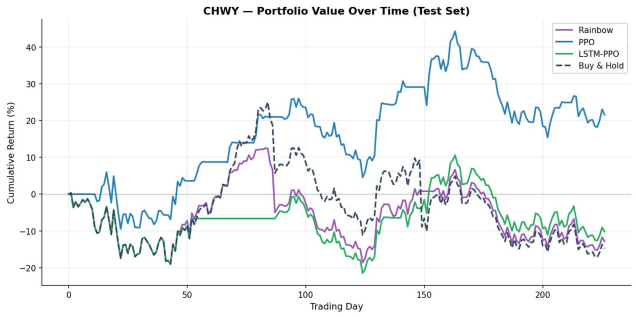

Figure 12: Portfolio Over Time for CHEWY

Three consistent findings emerge. First, PPO outperforms DQN across all settings, confirming that on-policy policy gradient methods with action masking and risk-aware rewards are better suited to trading than value-based approaches. Second, LSTM-PPO achieves the strongest performance on trending markets where temporal context aids regime detection, but underperforms on volatile, mean-reverting conditions, revealing sensitivity to distributional shift between training and test periods. Third, no single agent consistently beats buy-and-hold across all market conditions, highlighting that learned trading policies remain sensitive to the statistical properties of the evaluation period.

Conclusion

Single-asset trading from raw market data remains a fundamentally difficult problem: non-stationary dynamics, partial observability, and low signal-to-noise ratios ensure that no agent consistently dominates across all market regimes.

We presented a comparative study of four strategies Rainbow DQN, PPO, LSTM-A2C, and a Buy & Hold baseline for single-asset trading under the POMDP formulation, evaluated on AMZN and CHWY test sets.

PPO with event-based rewards proves to be the most robust agent across both tickers. On AMZN's trending test period, PPO achieves approximately +41% cumulative return, outperforming Buy & Hold (+29%) by 12 percentage points. On CHWY's declining market, PPO ends at roughly +22%, a remarkable result given that Buy & Hold finishes at 14%. This confirms that the event-based reward signal, which pe-

nalizes premature buys and rewards profitable round-trips, enables PPO to time entries and exits effectively.

LSTM-Actor Critic achieves +25% on AMZN (slightly below Buy & Hold), and approximately 10% on CHWY — closely tracking the market decline but unable to short or sidestep it sufficiently. While the LSTM encoder provides useful sequential context on trending markets, its performance on CHWY reveals sensitivity to distributional shift between training and test regimes: temporal patterns learned during training do not fully generalize to differing volatility conditions.

Rainbow DQN achieves a competitive +22% on AMZN, nearly matching LSTM-Actor Critic, but declines to approximately 12% on CHWY — performing similarly to Buy & Hold in the bearish regime rather than exploiting it. This suggests Rainbow's distributional value estimation and noisy exploration provide an advantage on trending assets, but its value-based structure struggles without the adaptive policy updates that PPO and LSTM-Actor Critic employ.

Three conclusions emerge. First, framing trading as a POMDP rather than an MDP motivates architectural choices LSTM belief approximation, action masking that measurably improve performance on trending markets. Second, reward engineering is a critical design axis: event-based rewards that align incentives with profitable round-trips outperform dense portfolio-value signals across all agents. Third, generalization across market regimes remains an open challenge; all agents lag PPO on CHWY, where the declining trend demands consistent sell discipline rather than opportunistic entry.

Future work includes training the LSTM end-to-end with the policy using gradient clipping strategies, incorporating multi-asset portfolios, enriching the observation vector with macroeconomic and sentiment features, and evaluating on longer out-of-sample horizons to better characterize regime generalization.

Disclosure

The authors used Claude (Anthropic) to assist with polishing and proofreading the writing in this paper. All research, experiments, code, analysis, and conclusions are the authors' own work.

Acknowledgments

This work was completed as part of a CS 5180: Reinforcement Learning (Spring 2026) course. The authors thank the course instructors for their guidance and feedback throughout the project.

References

Bellemare, M. G.; Dabney, W.; and Munos, R. 2017. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 449–458.

Fortunato, M.; Azar, M. G.; Piot, B.; Menick, J.; Osband, I.; Graves, A.; Mnih, V.; Munos, R.; Hassabis, D.; Pietquin, O.; et al. 2018. Noisy networks for exploration. In *International Conference on Learning Representations (ICLR)*.

Hausknecht, M.; and Stone, P. 2015. Deep reinforcement learning in parameterized action spaces. *arXiv preprint arXiv:1511.04143*.

Hessel, M.; Modayil, J.; Van Hasselt, H.; Schaul, T.; Ostrovski, G.; Dabney, W.; Horgan, D.; Piot, B.; Azar, M.; and Silver, D. 2018. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.

Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Computation*, 9(8): 1735–1780.

Kabbani, T.; and Duman, E. 2022. Deep reinforcement learning approach for trading automation in the stock market. *IEEE Access*, 10: 93564–93574.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; et al. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.

Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2016. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement learning: An introduction*. MIT Press, 2nd edition.

Van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30.

Wang, Z.; Schaul, T.; Hessel, M.; Van Hasselt, H.; Lanctot, M.; and De Freitas, N. 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1995–2003.

Yang, H.; Liu, X.-Y.; Zhong, S.; and Walid, A. 2020. Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the First ACM International Conference on AI in Finance (ICAIF)*, 1–8.